AutoAudit — Roteiro de apresentação do backend (Sprint 3)

03/11/2026, 19h · Fatec Cotia · PLN · Prof. Braz Izaias da Silva Junior Formato: sem slides; demonstração direto na aplicação rodando. **Duração:** 8 a 10 minutos.

1. Preparação (até 18h30)

Na raiz do repositório, nesta ordem:

1. `docker compose ps`: `mongo` e `mongo-express` em *running*.
2. `ls data/processed/`: `normas.index`, `normas_trechos.json`, `normas_index.json` presentes.
3. `python db/limpar_testes.py --apagar`: deixa só os exemplos do seed (3 aparecem no Histórico).
4. **Terminal 1:** `HF_HUB_OFFLINE=1 uvicorn backend.main:app` (**sem `--reload`**; o modo offline evita depender do Wi-Fi para carregar o modelo, que já está em cache).
5. **Terminal 2:** `streamlit run frontend/app.py`.
6. Abrir `http://localhost:8000/saude`: deve mostrar `api: ok`, `mongo: ok`, `busca_semantica: pronta` (aguarde até ~30 s).
7. **Auditoria de aquecimento** (a primeira carrega o modelo): envie qualquer PDF de `data/samples/` e depois rode `python db/limpar_testes.py --apagar` de novo.
8. Abas do navegador: app (8501), `/docs` (8000), mongo-express (8081). Os arquivos da demo numa pasta à mão.

Arquivos da demo

Arquivo	Onde	Resultado esperado
`Politica_Seguranca_Nimbus_Logistica.pdf`	`data/samples/`	nota 2,5; 6 lacunas
`Politica_Seguranca_Nimbus_Logistica_Revisada.docx`	`data/samples/`	nota 5,0; 0 lacunas
`modelo_politica_seguranca_informacao.pdf` (Governo Digital)	Downloads	nota 4,0; 9 lacunas
PoSIC UFPE / PoSIC IFB (reserva)	Downloads	3,9 / 3,7

Não use a PoSIC do SLU-DF (numeração inconsistente no PDF).

2. Sequência da demonstração

Tempo	Ação	O que dizer
0:00	Abrir o app na tela Nova auditoria	"O AutoAudit audita políticas de segurança da informação com PLN e dá uma nota de Saúde de Compliance de 0 a 5. Hoje mostro o backend da Sprint 3 funcionando de ponta a ponta, não simulado."
0:30	Mostrar a barra lateral e `/saude`	"A barra lateral mostra o estado real: API, MongoDB e busca semântica. É a mesma informação que a rota `/saude` devolve."
1:00	Enviar a **Nimbus PDF**	"A Nimbus é uma política fictícia que escrevemos com falhas de propósito. O motor extrai o texto, divide em seções, aplica regras e consulta a base de normas."
1:40	Resultado: nota 2,5 e abrir as 3 lacunas altas (senhas, acesso remoto, incidentes)	"Cada lacuna mostra o trecho original, o problema, a norma de referência (NIST e LGPD), a técnica MITRE quando existe e uma sugestão de nova redação. Aqui: 'senhas seguras' não é mensurável; acesso remoto sem MFA; incidentes sem prazo, responsável nem comunicação à ANPD."
3:00	Apontar "Normas próximas (busca semântica)" numa referência	"Além da referência das regras, o índice FAISS com 1.705 trechos de NIST e LGPD sugere normas próximas do trecho."
3:30	Enviar a **Nimbus revisada** (.docx)	"Reescrevemos a política seguindo as sugestões do relatório: 12 caracteres e bloqueio, MFA, prazo e responsável para incidentes, DPO."
4:10	Nota 5,0 e nenhuma lacuna	"A nota sobe de 2,5 para 5,0. O relatório é acionável: dá para corrigir e provar a melhora."
4:40	Enviar o **modelo do Governo Digital** (documento real)	"Agora um documento real, público, de 24 páginas, que não escrevemos. O motor remove cabeçalhos e rodapés, reconhece as seções e acha lacunas verificáveis."
5:30	Nota 4,0 e abrir 1 ou 2 lacunas	"É um estilo normativo de órgão público, então a nota é indicativa. Vê-se que as lacunas são médias e baixas, como 'periodicamente' e 'adequado' sem critério."
6:15	Aba Histórico; escolher a Nimbus	"O histórico vem do MongoDB, não da sessão do navegador. Por política, o gráfico mostra a evolução da nota ao longo do tempo."

	PDF	
6:45	mongo-express: `auditorias`, `relatorios`	"Cada auditoria tem estado (pendente, processando, concluída ou erro) e o relatório gravado, ligado às normas da coleção `normas`."
7:30	`/docs` do FastAPI	"A API é documentada automaticamente: `POST /auditorias`, `GET /relatorios`. O frontend só consome o contrato."
8:15	Limites e próximos passos	"Hoje é regra e busca semântica, sem LLM. A **Verificação Prática**, o pentest controlado em laboratório, **ainda não foi executada**: a nota é só documental. É a próxima etapa."

3. Como funciona (para explicar se pedirem)

- **Pipeline:** arquivo → texto (pypdf / python-docx) → seções e sentenças → regras → lacunas → nota → busca semântica de normas → grava no MongoDB.
- **Regras (regex):** senha, MFA em acesso remoto, incidente (prazo, responsável, ANPD), declaração genérica de legislação, dispositivo pessoal, ausência de DPO e termos vagos ("sempre que possível", "periodicamente", "quando necessário").
- **Nota documental:** pesos alta = 3, média = 2, baixa = 1. Fórmula: `5 × (1 − peso_total / (3 × unidades))`, mínimo 0. Unidades = maior entre o número de seções, 5 e 1 a cada 10 frases (para documento longo não ser punido só pelo tamanho). Saúde de Compliance = nota documental enquanto não há Verificação Prática; depois, média das duas.
- **FAISS:** 1.705 vetores (768 dimensões), modelo `paraphrase-multilingual-mpnet-base-v2`, NIST SP 800-53 + LGPD. Filtro por família de controle para cortar ruído.
- **MongoDB:** coleções `normas`, `auditorias` e `relatorios`, validador `$jsonSchema`, índices únicos e regras de integridade em `db/repositorio.py`.
- **Validação:** gabarito da Nimbus com 6 lacunas plantadas: 6 de 6 achadas, 0 falsos positivos; versão revisada: 0 lacunas.

4. Perguntas prováveis (respostas curtas)

Por que regras e não um LLM? Regras dão resultado determinístico e explicável: cada lacuna aponta a frase e o critério que disparou. É coerente com PLN baseado em padrões linguísticos. Um LLM é evolução natural, usando o FAISS como recuperação (RAG).

Como a nota é calculada? Está certa? É uma heurística explicável (pesos por severidade, normalizada pelo tamanho do documento). É provisória e foi calibrada nas políticas de teste; calibrar com mais documentos reais é o próximo passo.

Funciona com qualquer documento? PDF com texto e DOCX, até 10 MB, em português. PDF escaneado não: não há OCR, e o sistema avisa em vez de analisar errado.

Nota 5,0 significa conformidade total? Não. Significa que as regras atuais não acharam nada. As regras cobrem um conjunto limitado de problemas.

Para que serve o FAISS? Liga cada lacuna às normas mais próximas por similaridade semântica (embeddings), sem depender de palavra exata. Complementa as referências curadas nas regras.

E a Verificação Prática? Não foi executada nesta versão. `nota_pratica` e `verificacoes` voltam vazios e o frontend trata esse estado. Será o pentest controlado em ambiente de laboratório.

E a privacidade dos documentos? O processamento é local; nada é enviado a serviços externos. O banco guarda o hash do arquivo e o texto das lacunas.

O motor erra? Sim, e sabemos onde: títulos sem numeração, documentos normativos de órgãos públicos, PDFs com numeração inconsistente. Corrigimos falsos positivos em documentos reais que testamos hoje (definições, listas de normas, fragmentos).

O que a dupla fez? Banco de dados e modelagem (Sprint 2), frontend (Sprint 1) e agora o backend integrado: API, extração, regras, busca semântica e gravação no banco.

5. Se algo der errado (plano B)

Problema	O que fazer	O que dizer
API fora do ar	O app cai sozinho no modo simulado e mostra "Resultado SIMULADO". Subir a API de novo.	"Esse é o fallback que projetamos: o frontend funciona mesmo sem o backend, e avisa que o resultado é simulado."
Mongo fora do ar	`docker compose up -d` e reiniciar a API.	"A API devolve 503 quando o banco cai; o frontend usa o modo simulado."

Busca semântica lenta ou "carregando"	Esperar ~30 s. Se persistir, reiniciar a API sem `HF_HUB_OFFLINE=1` (precisa de internet).	"A primeira auditoria carrega o modelo de embeddings em memória; as seguintes são rápidas."
Primeira auditoria demora	Normal (modelo). Por isso o aquecimento.	—
Resultado inesperado num documento real	Seguir; abrir outra lacuna. Não improvisar correção ao vivo.	"O motor é baseado em regras e tem limitações; documentamos as principais."
Tela travada	`/docs` no navegador e mostrar `POST /auditorias` com o arquivo.	"Aqui o mesmo backend, sem o frontend."

6. Limitações assumidas (diga você antes de perguntarem)

- Regras por regex, sem LLM; só português.
- Só uma regra de ausência (Encarregado/DPO).
- Regra de senha aceita qualquer valor numérico mencionado, mesmo fraco.
- Títulos sem numeração só são reconhecidos em maiúsculas ou no estilo Título do Word.
- Documentos de órgãos públicos (PoSIC) têm estilo normativo; a nota é indicativa.
- **Verificação Prática ainda não implementada.**
- Demo local: o Streamlit Cloud não alcança a API nem o MongoDB do notebook.

7. Depois da apresentação

`git pull` (ver se o Moisés enviou algo) → `git push`. Atualizar o README com o status da Sprint 3.

8. Comandos de emergência

```
docker compose up -d                              # sobe o Mongo
HF_HUB_OFFLINE=1 uvicorn backend.main:app         # API (sem --reload)
streamlit run frontend/app.py                     # frontend
python db/limpar_testes.py --apagar               # limpa o histórico de testes
python backend/analisar_arquivo.py arquivo.pdf    # testa um documento sem API nem banco
```